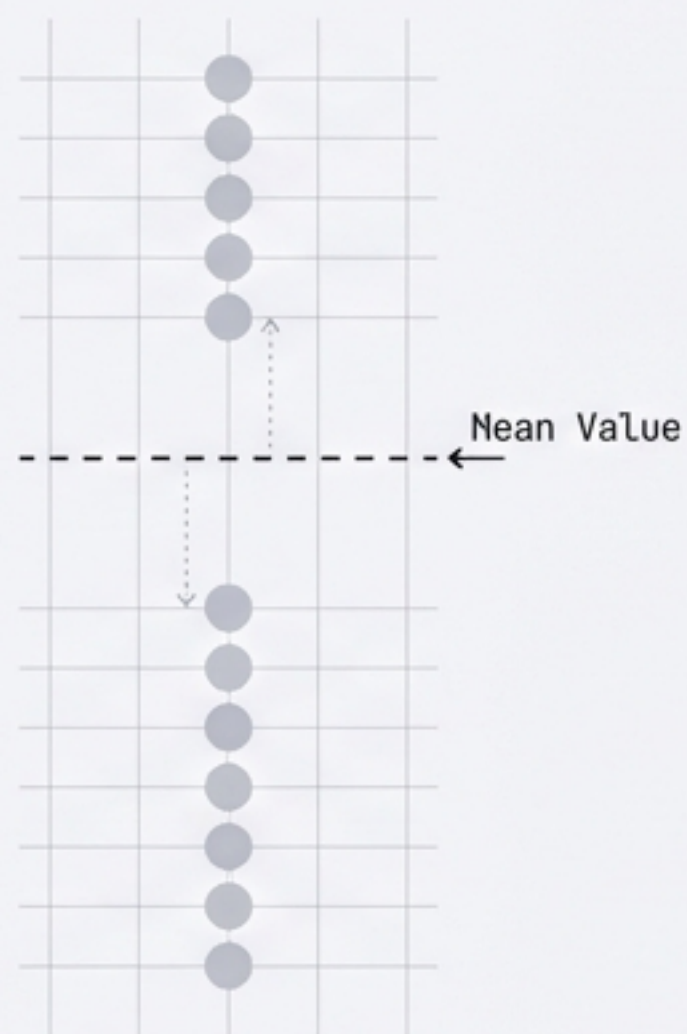


Beyond the Average: Intelligent Data Imputation with KNN

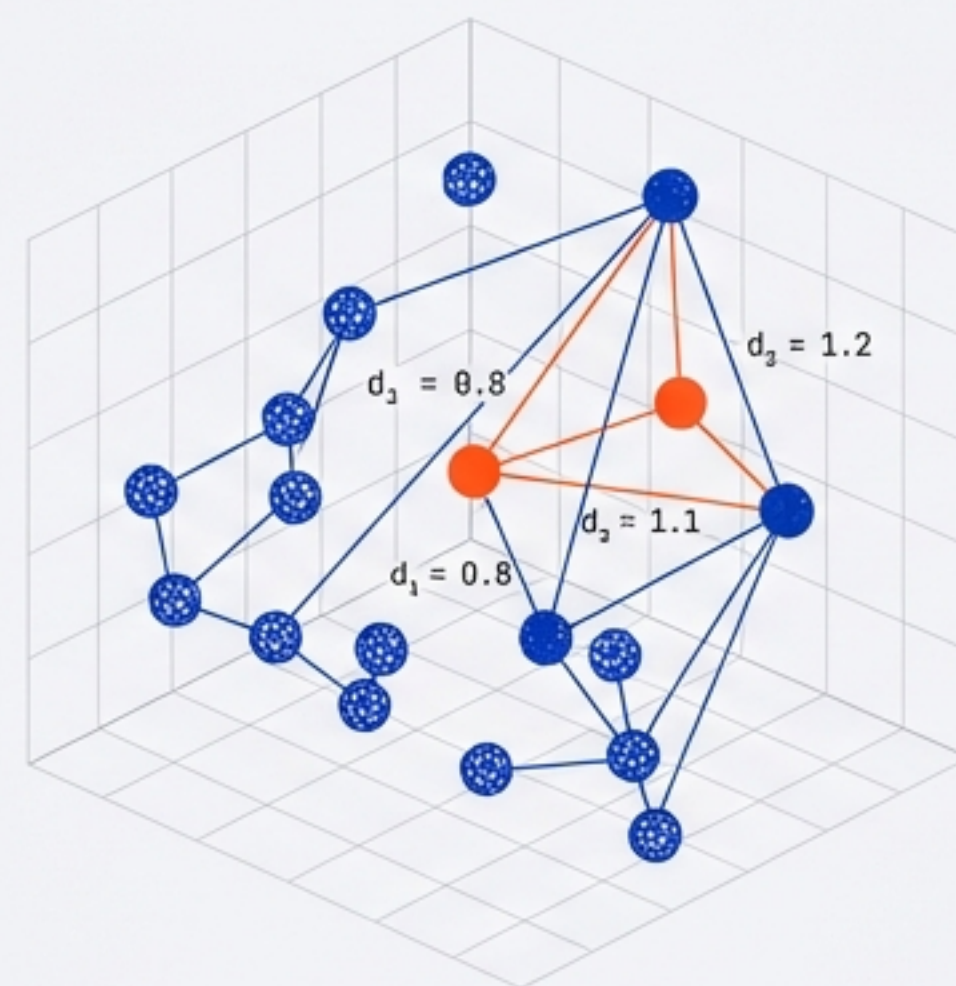
Moving from Univariate guessing to [Multivariate similarity](#).

The Lazy Approach



Univariate Imputation

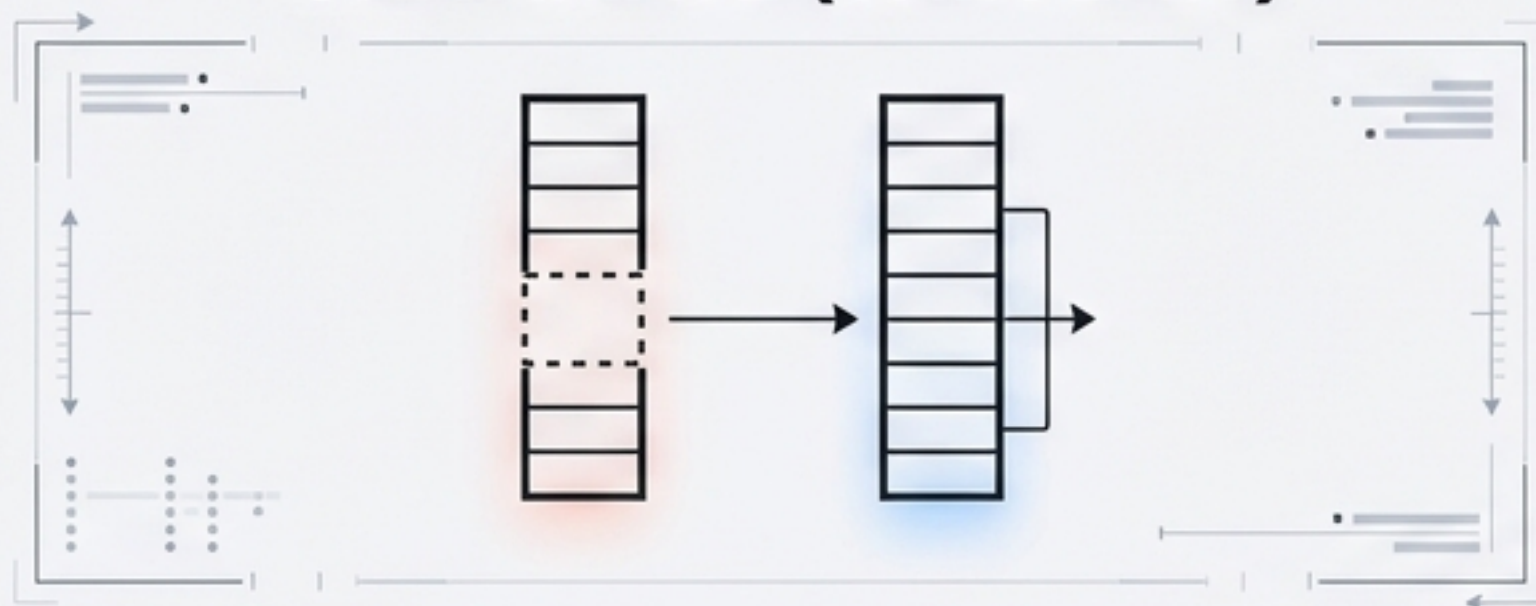
The Smart Approach



Multivariate KNN Imputation

THE LOGIC OF IMPUTATION: UNIVARIATE VS. MULTIVARIATE

UNIVARIATE (THE FLAW)

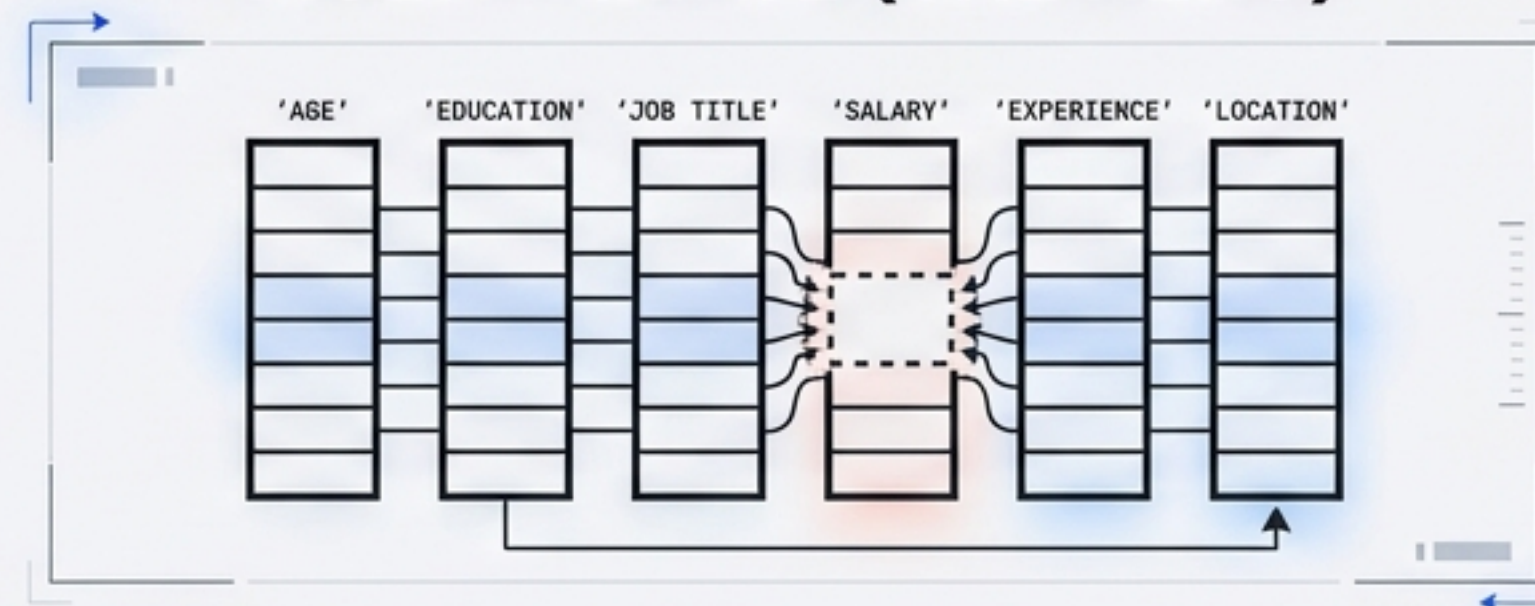


Fills missing values using only the distribution of that specific column.

METHODS: Mean, Median, Random Value.

THE FLAW: Ignores correlations. If you don't know a person's salary, guessing the national average ignores their specific age, education, and job title.

MULTIVARIATE (THE HERO)

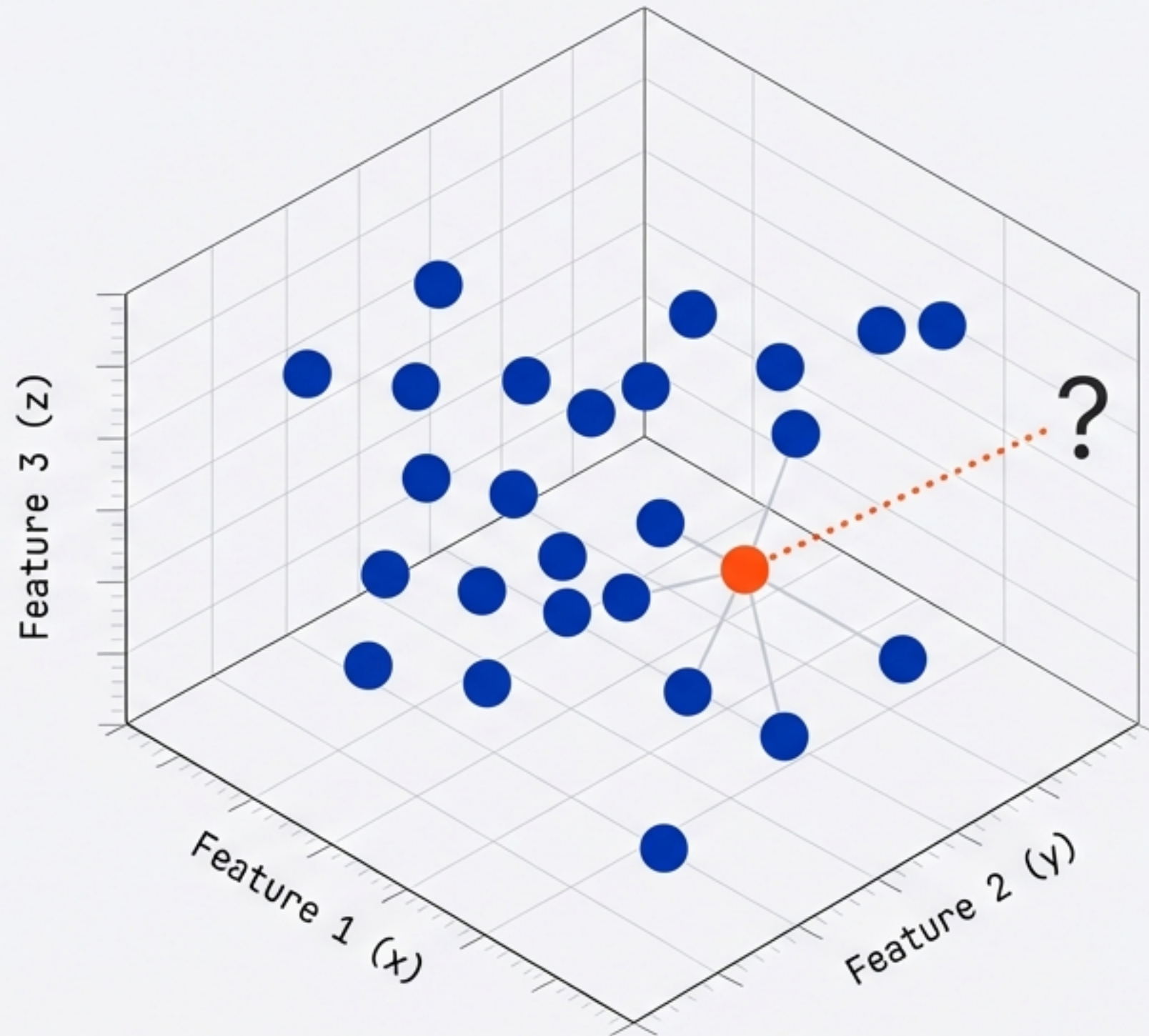


Uses the relationships between *all* columns (dimensions) to estimate the missing value.

THE HERO: K-Nearest Neighbours (KNN).

THE LOGIC: Finds the rows most similar to the one with the missing value, and uses their data to make an informed guess.

VISUALISING DATA AS GEOMETRY



Coordinate System:

Every row in your dataset is a coordinate in n -dimensional space.

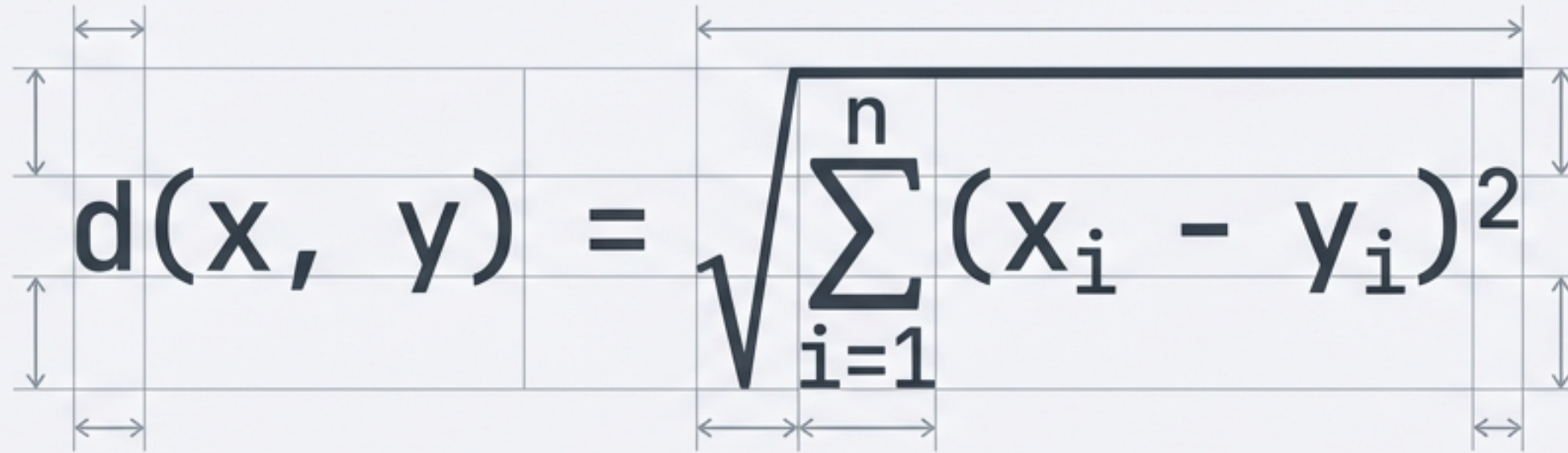
Physical Distance:

Similarity isn't abstract; it is the Euclidean distance between these points.

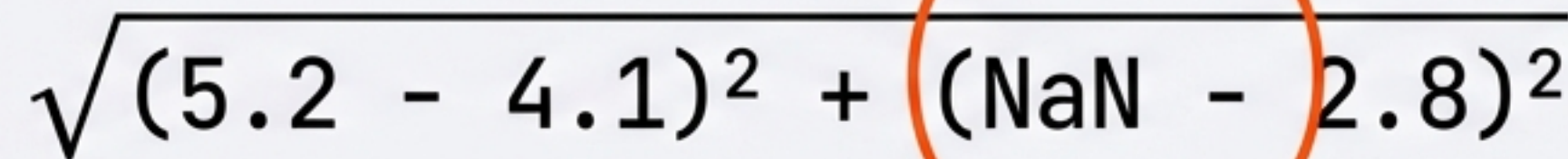
The Goal: To fill the missing z -value, we look at the z -values of the nearest neighbours.

THE MATHEMATICAL CHALLENGE

To find the “nearest” neighbour, we must calculate distance. The standard formula is Euclidean Distance:


$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

INSTANCE: ROW A VS. ROW B


$$\sqrt{(5.2 - 4.1)^2 + (\text{NaN} - 2.8)^2}$$

****Undefined Operation****

Standard math cannot subtract a missing value from a known number. **We need a modified distance metric.**

THE SOLUTION: NAN EUCLIDEAN DISTANCE

Adapting geometry to handle uncertainty.

$$\text{Distance} = \sqrt{\text{Weight} \times \sum_{i \in P} (x_i - y_i)^2}$$

Weight (The Correction Factor):

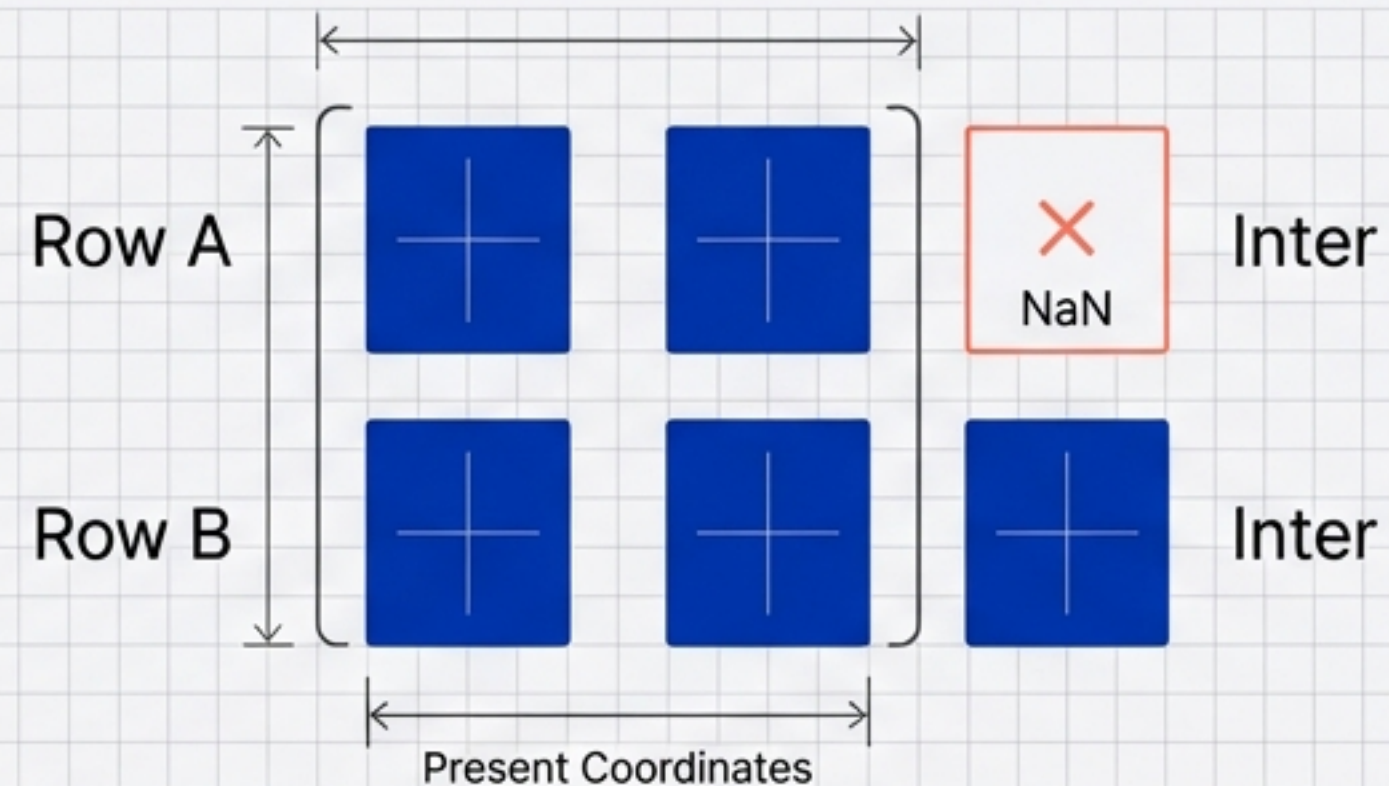
Scales the result to account for the missing dimensions. If we didn't scale, rows with fewer common points would artificially appear closer.

P :

The set of coordinates where *both* rows have valid values. We ignore the missing dimensions initially.

THE CORRECTION FACTOR (WEIGHT)

$$\text{Weight} = \frac{\text{Total Coordinates}}{\text{Present Coordinates}}$$



Total Coordinates (\$n\$): **3**

Present/Shared Coordinates: **2**

Weight Calculation: **\$3 / 2 = 1.5\$**

 We multiply the squared distance by **1.5**. This estimates what the total distance *would* have been if the third coordinate had been present, preventing bias towards sparse data.

WORKED EXAMPLE: CALCULATING DISTANCE

Top Section: The Data

Row 2 (Target):	[	55,	68,	NaN	]
Row 4 (Neighbour):	[	51,	61,	20	]

Step 1: Identify Common Ground

Row 2 (Target):	→	[	55,	68	]	NaN
Row 4 (Neighbour):		[	51,	61	]	20

Step 2: Calculate Weight

Total Columns = 3. Shared Columns = 2. Weight = **1.5**.

Step 3: The Equation Solved

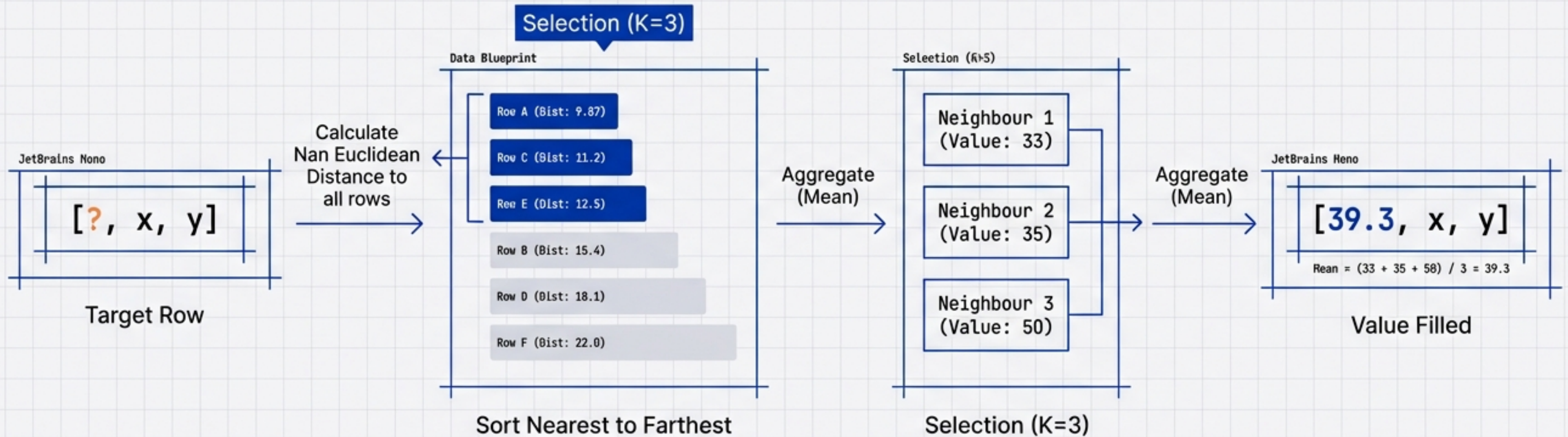
$$d = \sqrt{1.5 \times ((55 - 51)^2 + (68 - 61)^2)}$$

$$d = \sqrt{1.5 \times ((4)^2 + (7)^2)}$$

$$d = \sqrt{1.5 \times (16 + 49)} = \sqrt{97.5} \approx 9.87$$

The distance is derived mathematically, bypassing the missing value.

STEP 3: AGGREGATION (THE 'K' FACTOR)



REFINEMENT: UNIFORM VS. DISTANCE WEIGHTS



UNIFORM WEIGHTS (DEFAULT)

Democracy: Every neighbour gets 1 vote.

$$\frac{(70 + 50)}{2} = 60$$

Result is a simple average.



DISTANCE WEIGHTS (SMART)

Meritocracy: Closer neighbours have more influence.

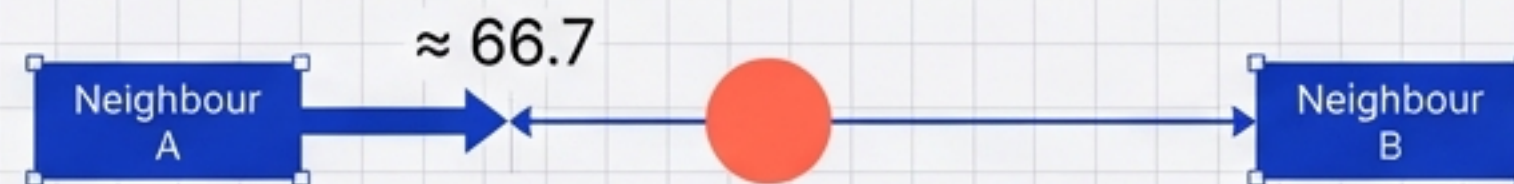
Weighted Average (using $1/d$):

$$w_A = 1/10 = 0.1$$

$$w_B = 1/50 = 0.02$$

$$\text{Result} = (70 \times 0.1 + 50 \times 0.02) / (0.1 + 0.02)$$

$$\text{Result} = (7 + 1) / 0.12 = 8 / 0.12 \approx 66.7$$

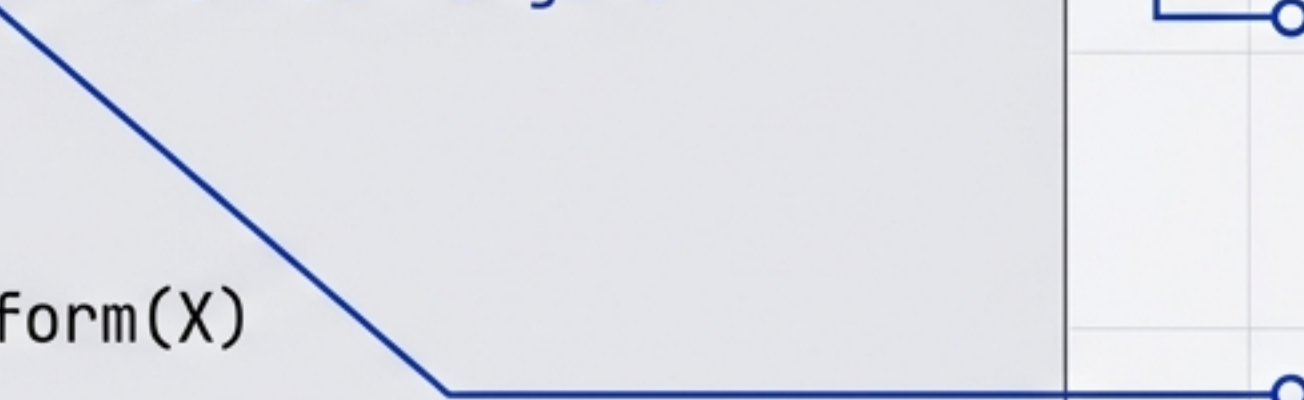


Result is pulled towards the more similar neighbour.

Implementation in Scikit-Learn

```
from sklearn.impute import KNNImputer
```

```
# Define the imputer
```

```
imputer = KNNImputer(  
    n_neighbors=5,     
    weights='distance',     
    metric='nan_euclidean'     
)
```

```
# Apply to data
```

```
X_trans = imputer.fit_transform(X)
```

 **n_neighbors:** The 'K' value. Needs experimental tuning (e.g., GridSearch).

 **weights:** Set to 'distance' to penalise far-away neighbours.

 **metric:** Defaults to 'nan_euclidean'. Handles the missing value math automatically.

Case Study: The Titanic Dataset

Pclass	Fare	Age
3	7.25	22
1	71.28	38
3	8.05	NaN
1	53.10	NaN

Missing Values

Experiment Setup

- **Goal:** Predict survival accuracy using Logistic Regression.
- **Challenge:** The Age column has significant missing data.
- **Hypothesis:** Age is correlated with Pclass (Wealth) and Fare. A wealthy passenger is likely older than a standard 3rd class passenger. KNN should capture this.

Performance Comparison



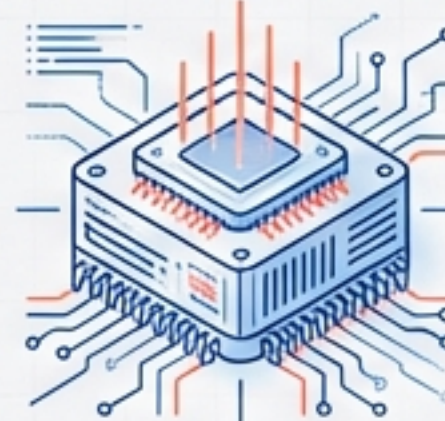
KNN outperforms univariate methods in this experiment. It successfully leveraged the correlation between `Fare/`Class` and `Age` to make smarter estimates, rather than just guessing the global average age.

The Trade-offs: Cost vs. Accuracy



The Upside (Pros)

- **Accuracy:**
Captures local data structure and non-linear relationships.
- **Utility:**
Effective for small to medium datasets.



The Downside (Cons)

- **Computation Speed:**
Slow. $O(N^2)$. Calculates distance between **every** point pair.
- **Memory Hog:**
Lazy Learner. Requires storing the **entire training dataset** in memory during production to find neighbours.

Summary & Best Practices

	Use when datasets are small to medium-sized.
	Use when features have strong correlations (e.g. Salary $\leftrightarrow$ Experience).
	Always scale data (MinMaxScaler or StandardScaler) before using KNN. Euclidean distance is sensitive to magnitude.

Data imputation isn't just about filling gaps; it's about preserving the relationships in your data.